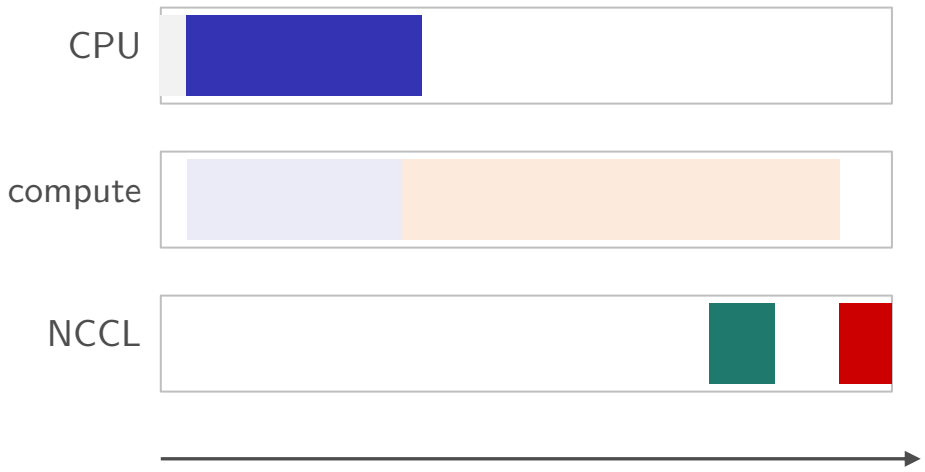
Diagnosing Distributed Systems

Reading timelines, finding stragglers, counting useful FLOPs and joules, and surviving failures

Yuke Wang

Rice University

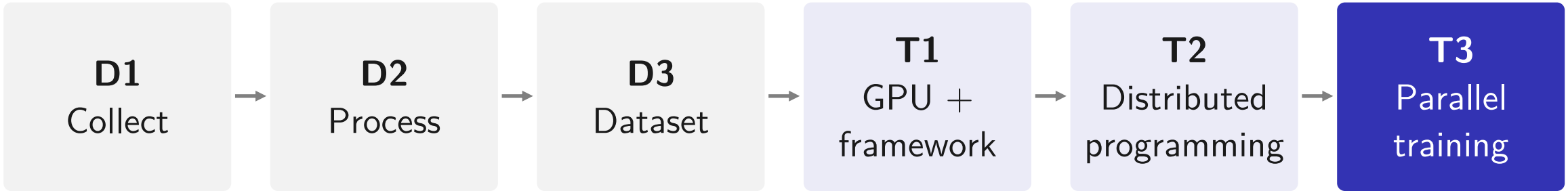
COMP 329 · Spring 2027 · Week 13, Lecture 2



one training step on one GPU (schematic)

blue: forward · orange: backward
green: all-reduce · red: exposed

Where we are in the pipeline



Lecture 1: pipeline parallelism, d $\times$ t $\times$ p layouts, a cost model

In our pipeline

Today: when the real run is slower than the model, **find out why** — and keep the run alive.

This week: **capstone pairs** are set. Week 14: integration and review; **T3 due**.

Learning goals for today

- Read a profiler timeline: compute vs **communication**, **exposed** vs overlapped, idle gaps.
- Explain why the **slowest rank** sets the step time, and detect stragglers per rank.
- Compute **MFU** and contrast it with "GPU utilization %".
- Measure **power and energy** per step and per token.
- List the state an **exact restart** needs; choose a checkpoint interval.
- Design a **reproducible** scaling experiment.

A puzzle to start

nvidia-smi
GPU-Util: **100%**

A training job shows 100% GPU utilization. Is the GPU doing as much arithmetic as it can?

Key idea

No. It only means *some* kernel was running in every sample. One busy SM out of 132 already counts as 100%.

Source: NVIDIA System Management Interface (nvidia-smi) documentation; NVML API reference.

Part 1

Reading a profiler timeline

Where did the time in one training step go?

Recording a timeline

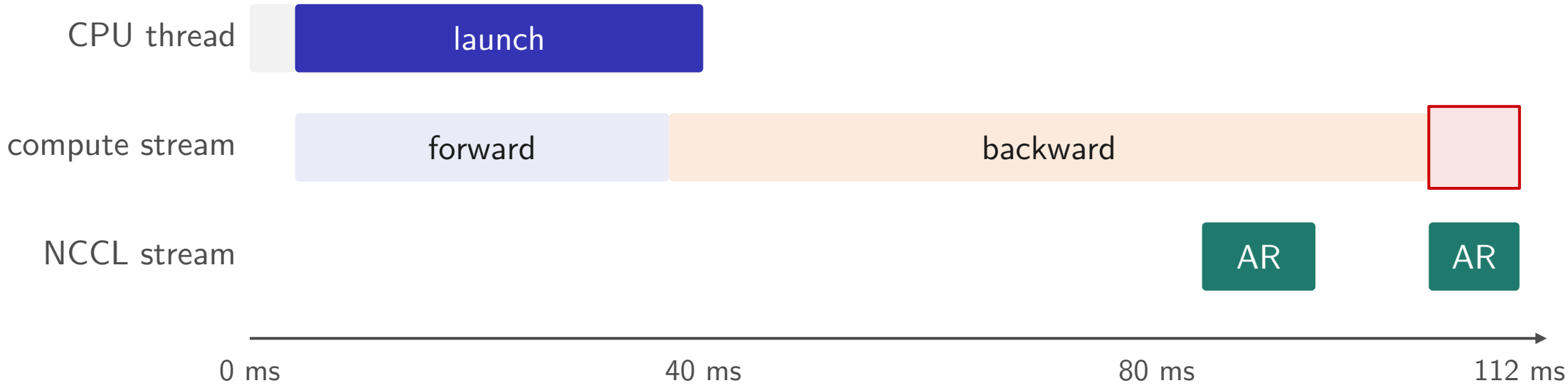
```
from torch.profiler import (profile,
    schedule, ProfilerActivity as A,
    tensorboard_trace_handler)

out = tensorboard_trace_handler(f"prof/r{rank}")
with profile(activities=[A.CPU, A.CUDA],
             schedule=schedule(wait=1, warmup=1,
                               active=3),
             on_trace_ready=out) as prof:
    for batch in loader:
        train_step(batch)
        prof.step()          # step boundary
```

- **One trace per rank**; open it in Perfetto.
- `wait`, `warmup`, `active`: skip start-up, then record a few steps.
- **Nsight Systems** (`nsys profile -t cuda,nvtx,osrt`): system-wide view.

Source: PyTorch documentation, torch.profiler; NVIDIA Nsight Systems User Guide.

Anatomy of a timeline



exposed: compute idle, NCCL busy

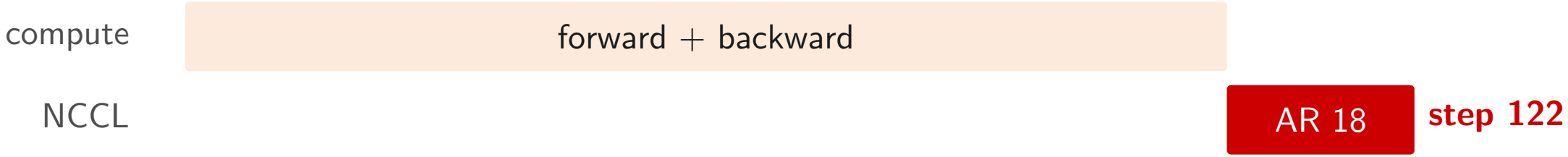
Key idea

Read the **compute** row first: every gap there is time you pay for.

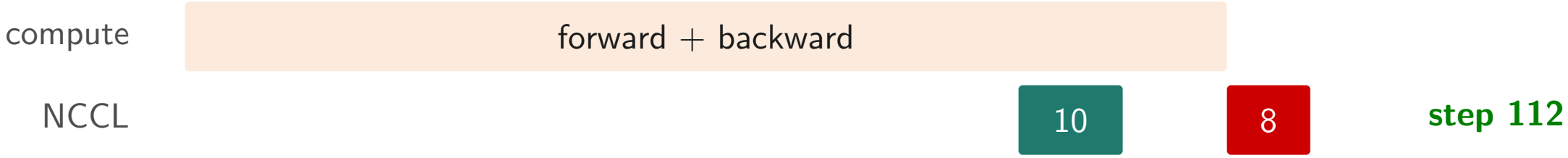
Source: PyTorch DDP documentation (gradient buckets, default `bucket_cap_mb` = 25).

Overlapped versus exposed communication

Reduce after backward



Bucketed (DDP): overlap with backward



Key idea

Overlapped communication is hidden under compute; only **exposed** communication lengthens the step.

Accounting for a step

$$T(\text{step}) = T(\text{compute}) + T(\text{exposed comm}) + T(\text{other idle})$$

$$\text{exposed} = \text{communication time} - \text{overlapped time}$$

$$112\ \text{ms} = 100 + (18 - 10) + 4$$

Watch out

A faster network can save **at most the exposed** 8 ms, not the full 18 ms.

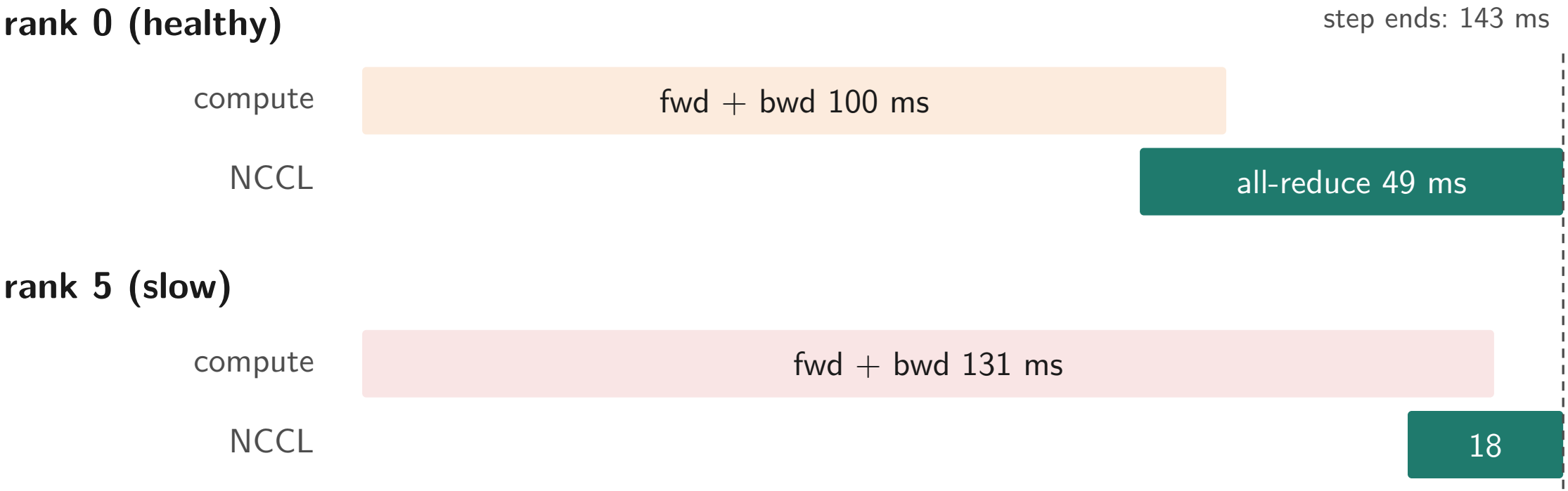
Idle gaps and what causes them

Gap in compute	Other rows show	Typical fix
Before forward	CPU in the data loader	more workers, prefetch
After a `.item()` or `print`	CPU waits on the GPU	log less often
Between tiny kernels	CPU busy launching	fuse, CUDA graphs, compile
After backward	a long NCCL kernel	overlap, or find the late rank

Key idea

Every gap has a cause visible in **another row** of the same timeline.

A wait looks like communication



Watch out

Same collective: 49 ms on rank 0, 18 ms on rank 5. Rank 0 is **waiting**.

Check yourself

Two minutes with a neighbour.

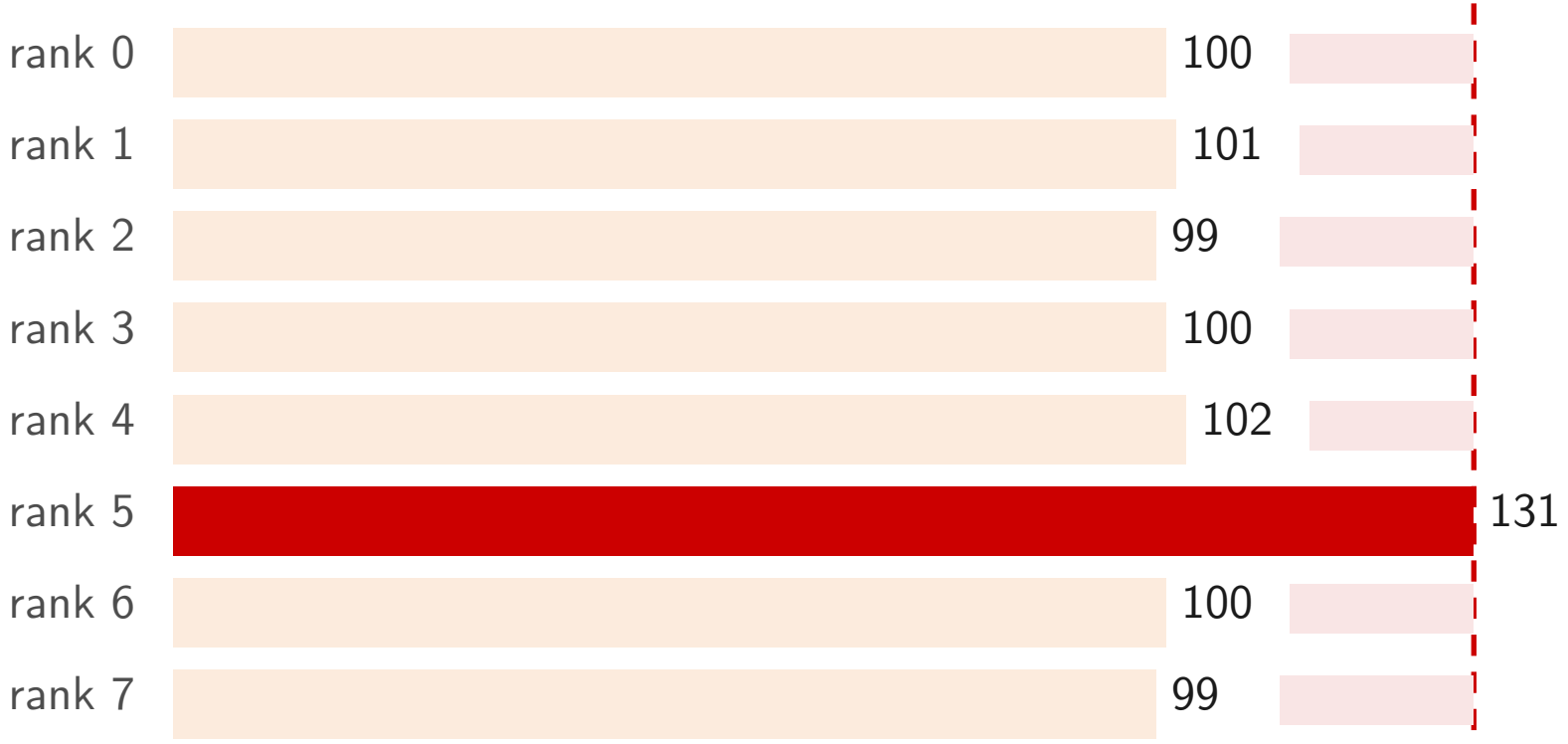
- Does every millisecond of NCCL kernel time add to the step time? Why not?
- Data 4 ms, compute 100 ms, all-reduce 30 ms with 10 ms overlapped: step time?
- Only rank 0's profile shows a 49 ms all-reduce. What do you check next?

Part 2

Stragglers

Why does one slow GPU out of a thousand matter?

The slowest rank sets the step



$$T(\text{step}) = \max \text{ over ranks } k \text{ of } T(k) + T(\text{comm})$$

Watch out

Seven ranks wait $\approx$ 30 ms each: **216 ms** of GPU time lost per step to one slow GPU.

One slow rank among many

$$P(\text{all } n \text{ ranks fast}) = (1-q) \cdot (1-q) \cdots (1-q)$$

$$P(\text{some rank slow}) \approx 1 - \exp(-q \cdot n)$$

q = 1%, n =	**8**	**64**	**512**	**1024**
P(slow step)	7.7%	47%	99.4%	$>$ 99.99%

Key idea

A **rare** event per GPU becomes an **every-step** event at scale.

The expected maximum grows with n

$$T(k) = 100 + 5 \cdot Z(k) \text{ ms}, \quad Z(k) \sim N(0, 1) \text{ independent}$$

n ranks	**1**	**8**	**64**	**1024**
E[max Z]	0	1.42	2.34	3.25
E[step] (ms)	100	107.1	111.7	116.2

Key idea

Noise alone costs **7–16%**. Cutting the **spread** σ helps as much as cutting the mean.

Why it grows so slowly

$$\exp(\lambda \cdot E[M]) \leq E[\exp(\lambda M)] \qquad \text{(Jensen)}$$

$$\leq \Sigma\ E[\exp(\lambda Z)] = n \cdot \exp(\lambda^2/2)$$

$$E[M] \leq \ln(n)\ /\ \lambda + \lambda\ /\ 2$$

$$\lambda = \text{sqrt}(2\ \ln\ n) \ \Rightarrow\ E[M] \leq \text{sqrt}(2\ \ln\ n)$$

Key idea

M = max of n standard normals. n = 1024: bound 3.72, exact 3.25 — slow growth, but **no ceiling**.

Detecting stragglers per rank

```
t0 = time.perf_counter()
local_forward_backward(batch)      # no comm
torch.cuda.synchronize()
mine = torch.tensor([time.perf_counter() - t0],
                    device=dev)
times = torch.empty(world, device=dev)
dist.all_gather_into_tensor(times, mine)
reduce_and_step()                  # all-reduce
slow = times.max() > 1.1 * times.median()
if rank == 0 and slow:
    print("straggler:", times.argmax().item())
```

- Time the **local** part: the full step is equal on every rank.
- Gather one number per rank; compare with the **median**.
- Persistent $\rightarrow$ hardware; occasional $\rightarrow$ noise.

Source: PyTorch documentation, torch.distributed: `all_gather_into_tensor`.

Causes and fixes

Cause	Evidence	Fix
Throttled GPU (heat, power cap)	lower clock and power	replace or cool
Slow link or NIC	one rank's sends are slow	move ranks, fix link
Uneven work (sequence lengths)	time tracks token count	pack, bucket by length
Host noise (GC, logging)	sporadic, often rank 0	pin CPUs, log less

Key idea

First question: is it **always the same rank**? Then it is the machine, not the code.

Check yourself

- Each of 256 ranks has a 1% chance of a slow step. How often is the job's step slow?
- Why is timing the whole step useless for finding the straggler?
- $T(k) = 100 + 2 \cdot Z(k)$ ms on 64 ranks: bound the expected step time.

Part 3

Utilization, energy, and scaling

How much of the hardware do we really use?

What “GPU utilization %” measures

nvidia-smi definition (paraphrased)

Percent of time over the past sample period during which **one or more kernels** was executing on the GPU.

- One kernel on **1 of 132 SMs** for the whole period → 100%.
- A memory-bound kernel that never uses tensor cores → 100%.
- A NCCL kernel **waiting** for a slow rank → 100%.

Watch out

Low utilization proves the GPU is idle; **high** utilization proves almost nothing.

Source: NVIDIA System Management Interface (nvidia-smi) documentation; NVML API reference.

Model FLOPs utilization (MFU)

n GPUs, each with peak π FLOP/s

$$\text{MFU} = (\text{model FLOPs per token} \times \text{tokens/s}) \;/\; (n \cdot \pi)$$

$$\text{dense transformer: } \approx 6\Psi \text{ FLOPs per token}$$

$$= 2\Psi \text{ forward} + 4\Psi \text{ backward}$$

Key idea

MFU counts only the FLOPs the **model** needs. Recomputation counts in **hardware** FLOPs (HFU), not MFU.

Source: Chowdhery et al., "PaLM," 2022 (MFU); Kaplan et al., 2020 ($\approx 6\Psi$ FLOPs per token); Narayanan et al., SC 2021, §5.1.

MFU, worked twice

40B model, 64 H100s

$6 \cdot 40\text{B} \cdot 131{,}072 \,/\, (64 \cdot 1.44\ \text{s})$

$= 341$ TFLOP/s per GPU

MFU $\approx$ **34%** of BF16 peak

Course MLP, one GPU

$6 \times 28{,}800$ weights $\times\ 64 = 11.1$ MFLOP

at 100 µs per step: 0.11 TFLOP/s

MFU $\approx$ **0.17%** of FP32 peak

Key idea

Large, well-parallelized models reach **30–50%** MFU; small, overhead-bound ones reach well under 1%.

Source: H100 datasheet (peaks); step times are the predictions of Lecture 1 and Week 8.

Power and energy

$$E = \int P(t)\ \mathrm{dt} \approx \Sigma\ P(k) \cdot \Delta t$$

$$\text{energy per token} = E(\text{step}) \ / \ \text{tokens per step}$$

$$64 \times 600 \text{ watts} \times 1.44 \text{ seconds} \approx 55 \text{ kJ per step} \ \rightarrow \ 0.42 \text{ joules per token}$$

Key idea

H100 SXM: up to **700 W** each. Report GPU energy **and** say what it excludes (CPUs, network, cooling).

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). 600 W average is an assumption.

Measuring power and energy

```
import pynvml as nv                    # nvidia-ml-py
nv.nvmlInit()
h = nv.nvmlDeviceGetHandleByIndex(local_rank)

e0 = nv.nvmlDeviceGetTotalEnergyConsumption(h)
for _ in range(100):
    train_step()
torch.cuda.synchronize()
e1 = nv.nvmlDeviceGetTotalEnergyConsumption(h)
print((e1 - e0) / 1000 / 100, "J per step")
print(nv.nvmlDeviceGetPowerUsage(h) / 1000, "W")
```

- Energy counter: **millijoules**, cumulative.
- Power: **milliwatts**, current.
- `nvidia-smi` (`power.draw`) for a quick look.

Source: NVIDIA System Management Interface (nvidia-smi) documentation; NVML API reference.

Strong and weak scaling

	Strong scaling	**Weak scaling**
Held fixed	global batch (total work)	batch per GPU
Ideal	$T(n) = T(1) / n$	$T(n) = T(1)$
Speedup	$S(n) = T(1) / T(n)$	throughput ratio
Efficiency	$S(n) / n$	$T(1) / T(n)$

In our pipeline

T3 asks for **strong** scaling: same model, data, global batch 64, seed, and steps.

Amdahl revisited

$$T(n) = T(1) \cdot \left((1 - f) \,/\, n + f \right)$$

$$S(n) \to 1 \,/\, f \quad \text{as} \quad n \to \infty$$

Watch out

4% that does not shrink caps the speedup at **25×**; fit f to your T3 timings.

n	T (ms)	S	E
1	1000	1.00	100%
2	520	1.92	96%
4	280	3.57	89%
8	160	6.25	78%
16	100	10.00	62%
64	55	18.18	28%

f = 4%, T(1) = 1000 ms

Source: Amdahl, AFIPS 1967; the f = 4% example is illustrative.

Check yourself

- $\Psi = 7$B, one H100 processes 4,000 tokens/s in training. MFU (BF16 peak)?
- 8 GPUs at 650 W, step 0.5 s, 64,000 tokens per step: energy per token?
- T(1) = 400 ms, T(8) = 60 ms. Speedup, efficiency, and Amdahl's f?

Part 4

Checkpointing and recovery

How do we restart a job exactly where it stopped?

Failures are routine at scale

$$\mathrm{MTBF}(\text{job}) = \mathrm{MTBF}(\text{component}) \;/\; n$$

Llama 3 pre-training

54 days, 16,384 H100s: **419** unexpected interruptions $\rightarrow$ one every $\approx$ **3.1 hours**.



Key idea

On failure: **all** ranks restart from the last **complete** checkpoint and redo the lost work.

Source: Llama Team, Meta, "The Llama 3 Herd of Models," 2024, §3.3.4.

What an exact restart needs

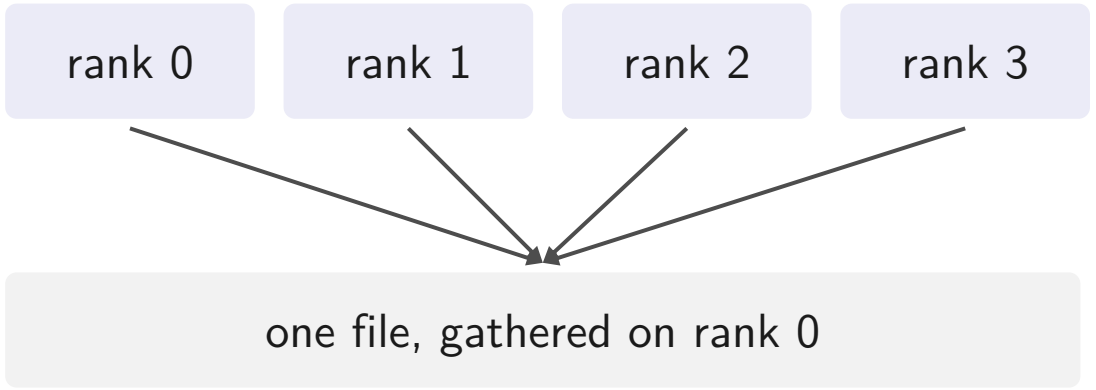
State	Forget it and ...
Model parameters	training starts over
Optimizer state (Adam m, v, step)	updates jump; bias correction restarts
LR scheduler	warm-up repeats; wrong learning rate
RNG states, all sources, every rank	different dropout and shuffles
Data loader position, sampler epoch	data repeated or skipped
Step and token counters	logs and schedules drift

Check

Test it: train 2k steps; separately train k, stop, restart, train k — losses must **match**.

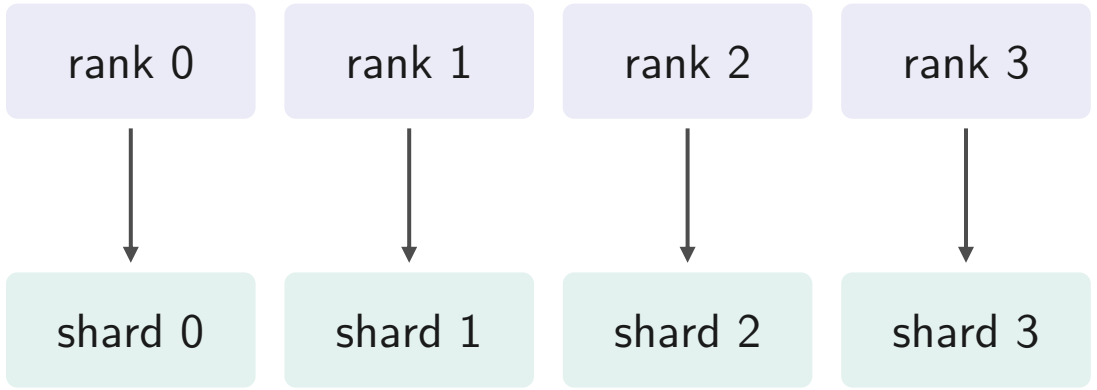
Sharded versus consolidated checkpoints

Consolidated



needs all state on rank 0; one writer

Sharded (DCP)



parallel writes + a .metadata file; reload on a different n

Key idea

At scale, save **sharded**: every rank writes its own piece, in parallel.

Source: PyTorch documentation: torch.distributed.checkpoint (DCP). Narayanan et al., SC 2021, §5.10.

Distributed Checkpoint (DCP) in code

```
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint import (
    state_dict as sd)

m, o = sd.get_state_dict(model, opt)
state = {"model": m, "optim": o}
dcp.save(state, checkpoint_id=f"ckpt/{step}")

# restart: same structure, filled in place
m, o = sd.get_state_dict(model, opt)
state = {"model": m, "optim": o}
dcp.load(state, checkpoint_id=f"ckpt/{step}")
sd.set_state_dict(model, opt,
    model_state_dict=m, optim_state_dict=o)
```

- **Every rank** calls `save` and `load` (collective).
- Add scheduler, step, RNG to `state` (Stateful objects).
- `dcp.async_save` overlaps the write with training.

Source: PyTorch documentation: torch.distributed.checkpoint (DCP).

Consistency: one step, all or nothing

1. write
ckpt/tmp-1000/

→

2. barrier
all ranks done

→

3. rename
→ ckpt/step-1000/

→

4. publish
latest → step-1000

- All ranks save the **same step** (a collective at a step boundary).
- Rename within one file system is **atomic**: never a half-written name.
- Restart from what `latest` points to; keep the last **k** checkpoints.

Key idea

A checkpoint is either **complete and published**, or invisible.

How often to checkpoint: Young's approximation

$$\text{overhead}(\tau) \approx \delta \;/\; \tau + \tau \;/\; (2M)$$

$$d/\mathrm{d}\tau\text{:}\;\; -\delta \;/\; \tau^2 + 1 \;/\; (2M) = 0$$

$$\tau^* = \text{sqrt}(2\delta M), \quad \text{overhead}^* \approx \text{sqrt}(2\delta \;/\; M)$$

Watch out

δ = blocking write time, M = job MTBF. A **first-order approximation**: valid when $\delta \ll$ M.

Source: J. W. Young, "A first order approximation to the optimum checkpoint interval," CACM 17(9), 1974.

Young's rule, worked

$M = 3.1$ h (Llama 3 rate); δ values are assumptions

	δ (blocking write)	interval τ^*	overhead
Synchronous save	60 s	$\approx$ 19 min	$\approx$ 10%
Asynchronous save	5 s	$\approx$ 5.6 min	$\approx$ 3%

Key idea

Overhead grows like the **square root of** δ: shrink the blocking part, then save often.

Source: Young, CACM 1974; Llama 3 paper, 2024 (interruption rate).

Reproducible scaling experiments

Control	How
Seeds	`torch.manual_seed`, `random`, `numpy`, loader workers
Deterministic kernels	`use_deterministic_algorithms(True)`; cuBLAS env var
Same optimization	fixed **global** batch, data order, steps
Statistics	warm up; ≥ 3 repeats; **median** and spread
Environment	GPU, driver, CUDA, NCCL, PyTorch, topology

Watch out

Different n changes the all-reduce's **addition order**: expect agreement in statistics, not in bits.

Source: PyTorch documentation, Reproducibility notes.

Check yourself

- Why must the checkpoint include the sampler's epoch and position?
- $\delta = 30$ s, job MTBF $= 10$ hours: checkpoint interval and overhead?
- What goes wrong if each rank writes directly to `ckpt/latest/`?

IN-CLASS BUILD

Diagnose a slow job

Work in pairs. Read the numbers; propose one fix.

Exercise: one step, eight ranks

Rank	Data	Compute	NCCL	Power
0	4	100	49	650
1	4	101	48	650
2	4	99	50	650
3	4	100	49	650
4	4	102	47	650
5	4	131	18	500
6	4	100	49	650
7	4	99	50	650

times in ms, power in W · every rank's step: 143 ms

GIVEN

All-reduce alone: 18 ms,

10 ms of it overlapped

$\Psi = 1$B; 8,192 tokens per rank

BF16 peak 989 TFLOP/s

FIND

1. Straggler? Evidence?
2. Exposed comm?
3. MFU? 4. Energy per token?
5. One fix + gain

Solutions: diagnosis

- **Rank 5** computes for 131 ms vs $\approx$ 100 ms elsewhere.
- Its all-reduce is the **shortest** (18 ms); the others wait $\approx$ 49 ms.
- It draws 500 W vs 650 W: likely **power-capped or throttling**.
- Exposed communication: $18 - 10 = \mathbf{8\ ms}$.

$$143\ \text{ms} = 4\ (\text{data}) + 131\ (\text{rank } 5) + 8\ (\text{exposed})$$

Key idea

Rank 0's 49 ms all-reduce = **31 ms waiting** + 8 exposed + 10 overlapped.

Solutions: MFU, energy, and a fix

	Now	After the fix
Step time	143 ms	114 ms
MFU	34.8%	43.6%
Energy per step	722 J	$\approx$ 593 J
Energy per token	11.0 mJ	$\approx$ 9.0 mJ

$$\mathrm{MFU} = 6\Psi \cdot \mathrm{tokens} \,/\, (8\pi \cdot T)$$

Fix: move off rank 5's GPU; the slowest remaining rank (102 ms) sets the step.

Prediction

Predicted **1.25×** faster. Perfect overlap would save at most 8 ms ($\approx$ 6%): **fix the straggler first**.

Key takeaways

- Read the **compute row** first; only **exposed** communication lengthens a step.
- A long collective on one rank usually means it is **waiting** for a straggler.
- Synchronous steps pay the **maximum**: rare per-GPU hiccups become every-step costs.
- Report **MFU** (6Ψ per token) and **energy per token**, not "GPU utilization %".
- Exact restart = params + optimizer + scheduler + **RNG** + **data position**, saved atomically.
- Checkpoint about every **sqrt(2δM)**; scale runs with fixed seeds and global batch.

Next time, and readings

When	What
This week	Capstone pairs are set
Week 14	Joint integration and review
Week 14	T3 due: DP vs DDP, TP/PP exercises, layout analysis, scaling study

READ

Narayanan et al., SC 2021: §5.1 and Eq. (3), how FLOP/s are counted.

PyTorch documentation: *Distributed Checkpoint* (torch.distributed.checkpoint).